Tutorial: Memoria persistente y servidores MCP en VS Code (Node + uv)

Guía paso a paso · Node.js · uv · VS Code · GitHub Copilot Chat

EN ESTA GUÍA

Paso 1: Instalar Node.js 2
Paso 2: Instalar uv 2
Paso 3: Instalar y preparar VS Code 3
Paso 4: Crear la carpeta del proyecto 4
Paso 5: Configurar el servidor de memoria 5
Paso 6: Probar la memoria 7
Paso 7: Crear tu propio servidor MCP en Python con uv 8
Paso 8: Registrar tu servidor en VS Code 10
Paso 9: Depurar con el MCP Inspector (opcional, pero muy útil) 12
Paso 10 (extra): Usar servidores de terceros con uvx 13
Solución de problemas comunes 13

Al terminar vas a tener dos cosas funcionando en VS Code:

1. **Un servidor de memoria** (el oficial de MCP, hecho en Node) para que el asistente recuerde datos tuyos entre conversaciones.
2. **Tu propio servidor MCP en Python**, creado y ejecutado con **uv**, que guarda notas en un archivo.

Los dos se conectan a **GitHub Copilot Chat en modo Agente**, que es el cliente MCP integrado en VS Code.

> **¿Qué es MCP?** Model Context Protocol (MCP) es un estándar abierto que permite a un asistente de IA usar herramientas externas: leer archivos, consultar una base de datos, guardar notas, etc. El asistente actúa como cliente y cada herramienta vive en un servidor MCP. Piensa en MCP como un «puerto USB-C» para la IA: cualquier servidor compatible se conecta a cualquier cliente compatible.

Cómo encaja todo

Este es el mapa de lo que vas a construir. Vuelve a él si en algún paso pierdes de vista qué pieza estás configurando:

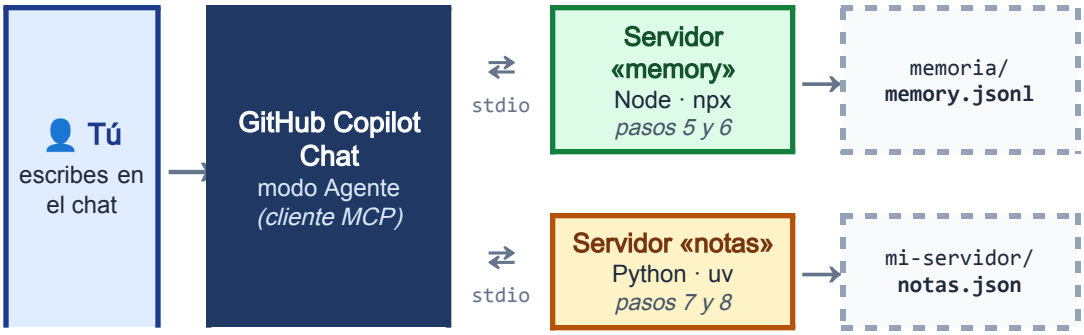


Figura 1. VS Code arranca cada servidor como un proceso y se comunica con él por stdio. Cada servidor guarda sus datos en su propio archivo.

Paso 1: Instalar Node.js

El servidor de memoria se ejecuta con npx, que viene con Node.js. Además, el inspector de MCP del paso 9 también usa Node.

Descarga la versión LTS desde https://nodejs.org e instálala con las opciones por defecto. La versión LTS es la más estable y la recomendada para este tutorial.

Verifica la instalación (en una terminal nueva):

```bash
node -v    # versión de Node.js
npm -v     # gestor de paquetes de Node
npx -v     # ejecutor de paquetes (lo usa el servidor de memoria)
```

Si los tres comandos muestran un número de versión, Node quedó bien instalado.

> **Importante:** si VS Code o la terminal ya estaban abiertos cuando instalaste Node, ciérralos y vuelve a abrirlos. Así reconocen el nuevo PATH y los comandos node y npx quedan disponibles.

Paso 2: Instalar uv

uv es un gestor de Python muy rápido. Instala Python, crea entornos virtuales y ejecuta scripts, todo en una sola herramienta.

Instálalo con el comando de tu sistema operativo:

```
PowerShell · Windows
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

```
Terminal · macOS / Linux
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Cierra y vuelve a abrir la terminal, y comprueba que quedó instalado:

```
bash
uv --version    # debe mostrar algo como «uv 0.x.x»
```

No necesitas tener Python instalado previamente. uv lo descarga cuando hace falta. Si quieres dejarlo instalado desde ya:

```
bash
uv python install    # opcional: descarga la última versión estable de Python
```

Paso 3: Instalar y preparar VS Code

1. Descarga VS Code desde https://code.visualstudio.com e instálalo. En Windows, marca "Agregar a PATH" y "Abrir con Code".
2. Abre VS Code, ve a **Extensiones** (Ctrl+Shift+X) e instala:
   - **GitHub Copilot Chat**. Con la cuenta gratuita de Copilot es suficiente para empezar.
   - **Python** (de Microsoft). Es opcional, pero ayuda con el código del paso 7.
3. Inicia sesión con tu cuenta de GitHub cuando VS Code te lo pida.
4. Abre el chat con Ctrl+Alt+I (en Mac, Cmd+Ctrl+I). Abajo, en el selector de modo, elige Agent. **Los servidores MCP solo funcionan en modo Agente.**

Consejo: si no ves la opción Agent en el selector de modo, actualiza VS Code y la extensión GitHub Copilot Chat a la última versión. El modo Agente es el único que puede llamar herramientas MCP; los modos Ask y Edit no.

Paso 4: Crear la carpeta del proyecto

Crea la estructura base desde la terminal:

```bash
# 1. Crea la carpeta raíz del proyecto
mkdir proyecto-mcp
# 2. Entra en ella
cd proyecto-mcp
# 3. Crea la carpeta de configuración de VS Code y la de la memoria
mkdir .vscode memoria
# 4. Abre la carpeta actual en VS Code (el punto significa «aquí»)
code .
```

Al final del tutorial el proyecto quedará así:

```
proyecto-mcp/
├── .vscode/
│   └── mcp.json          ← configuración de servidores MCP
├── .github/
│   └── copilot-instructions.md
├── memoria/
│   └── memory.jsonl      ← se crea solo
└── mi-servidor/          ← tu servidor Python (paso 7)
```

Nota: por ahora solo creas .vscode y memoria. Las carpetas .github y mi-servidor las crearás en los pasos 6 y 7, y el archivo memory.jsonl lo genera el servidor automáticamente la primera vez que guarda algo.

Paso 5: Configurar el servidor de memoria

Crea el archivo .vscode/mcp.json con este contenido:

```json · .vscode/mcp.json
{
  "servers": {
    "memory": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-memory"],
      "env": {
        "MEMORY_FILE_PATH": "${workspaceFolder}/memoria/memory.jsonl"
      }
    }
  }
}
```

Esto es lo que hace cada parte:

- **"type": "stdio":** VS Code arranca el servidor como un proceso y se comunica con él por la entrada y salida estándar.
- **npx -y ...:** descarga y ejecuta el paquete oficial de memoria. La opción -y evita que te pida confirmación.
- **MEMORY_FILE_PATH:** indica dónde se guarda la memoria. Si no lo defines, el archivo queda escondido dentro de la caché de npm y es difícil de encontrar.

Ojo: ${workspaceFolder} es una variable de VS Code que se reemplaza por la ruta de la carpeta que tienes abierta. Por eso es importante abrir VS Code dentro de proyecto-mcp (con code .) y no en una carpeta superior; si no, la memoria se guardará en otro lugar.

Arrancar el servidor: al guardar el archivo aparece un botón ▶ **Start** encima de "memory". Haz clic en él. Si arranca bien, verás "Running" y el número de herramientas disponibles.

```
.vscode > {} mcp.json > { } servers > { } memory
{
  "servers": {
    ✓Running | Stop | Restart | More...
    "memory": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-memory"],
      "env": {
        "MEMORY_FILE_PATH": "${workspaceFolder}/memoria/memory.jsonl"
      }
    }
  }
}
```

También puedes arrancarlo desde la paleta de comandos: Ctrl+Shift+P → **MCP: List Servers** → memory → Start.

Cómo funciona esta memoria

Guarda la información como un **grafo de conocimiento** con tres elementos:

- **Entidades:** por ejemplo, "Ana", de tipo persona.
- **Relaciones:** por ejemplo, "Ana trabaja_en Empresa X".
- **Observaciones:** datos sueltos sobre una entidad, como "Prefiere Python".

Para manejar ese grafo, el servidor ofrece estas herramientas: create_entities, create_relations, add_observations, search_nodes, open_nodes, read_graph, y sus equivalentes delete_*.

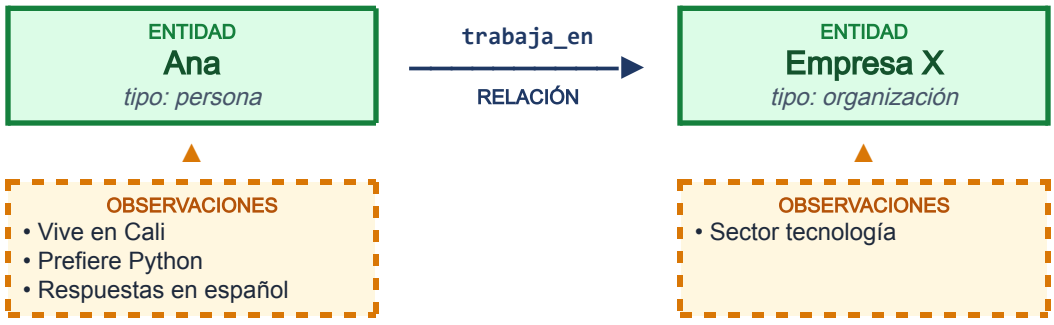
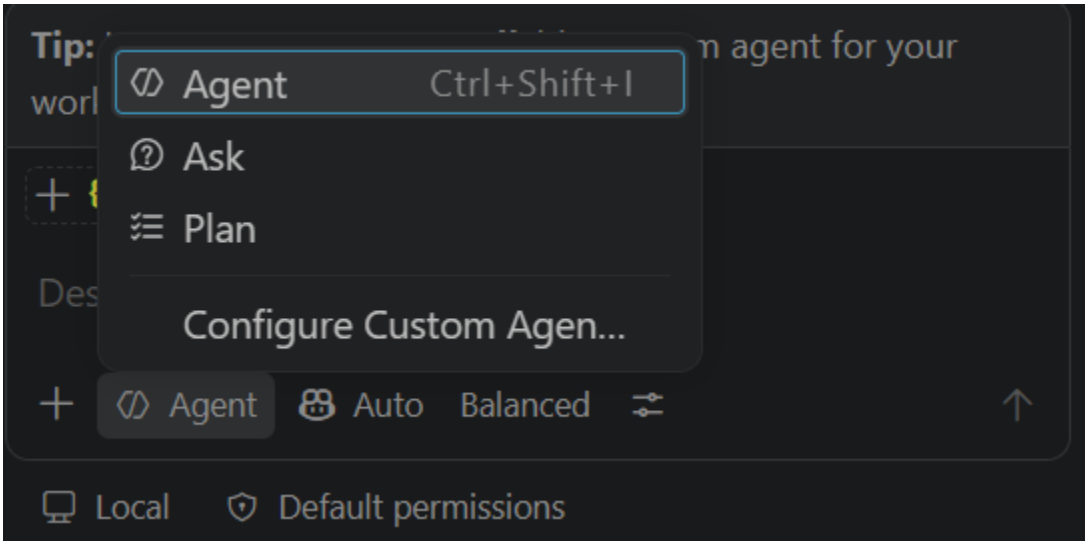


Figura 2. Ejemplo de grafo de conocimiento: dos entidades unidas por una relación, cada una con sus observaciones.

Paso 6: Probar la memoria

1. En el chat (modo **Agent**), haz clic en el icono de **herramientas** 🔧 y comprueba que las de "memory" estén activadas.



2. Escribe algo como:

   usa el mcp memory Recuerda que me llamo Ana, trabajo en Cali como desarrolladora y prefiero respuestas en español con ejemplos en Python.

3. Copilot te pedirá permiso para usar create_entities. Acepta. Puedes elegir "Allow for this session" para que no vuelva a preguntar.
4. Abre memoria/memory.jsonl y verás tus datos guardados en formato JSON.
5. Abre un **chat nuevo** (botón +) y pregunta:

   ¿Qué sabes de mí? Revisa tu memoria.

El asistente llamará a read_graph o a search_nodes y te responderá con lo que guardaste.

Privacidad: memory.jsonl es un archivo de texto plano en tu disco: cualquiera con acceso a la carpeta puede leerlo. Si el proyecto está en Git, agrega la línea memoria/ a tu .gitignore para no subir tus datos personales.

Hacer que use la memoria automáticamente

Para que no tengas que pedirle cada vez que consulte la memoria, crea el archivo .github/copilot-instructions.md:

```markdown
# Uso de memoria

- Al inicio de cada conversación, usa la herramienta
  `search_nodes` o `read_graph` del servidor "memory" para
  recuperar información relevante sobre el usuario.
- Cuando el usuario comparta datos personales, preferencias,
  proyectos o decisiones importantes, guárdalos con
  `create_entities`, `create_relations` o `add_observations`.
- No guardes contraseñas, tokens ni datos sensibles.
```

VS Code añade estas instrucciones a todas las conversaciones de ese proyecto. Si no se aplican, revisa que el ajuste github.copilot.chat.codeGeneration.useInstructionFiles esté activado.

Paso 7: Crear tu propio servidor MCP en Python con uv

Ahora vas a crear un servidor de notas sencillo.

Desde la raíz del proyecto:

```bash
uv init mi-servidor    # crea el proyecto Python
cd mi-servidor         # entra en la carpeta del servidor
uv add "mcp[cli]"      # crea .venv e instala el SDK de MCP
```

Esto hace dos cosas:

- **uv init** crea pyproject.toml, main.py y la configuración básica del proyecto.
- **uv add** crea el entorno virtual .venv e instala el SDK oficial de MCP.

Borra main.py y crea server.py con este contenido:

```python
import json
from datetime import datetime
from pathlib import Path

# FastMCP es la forma más sencilla de crear un servidor con el SDK oficial
from mcp.server.fastmcp import FastMCP
```

```python
# Nombre con el que el servidor se presenta al cliente
mcp = FastMCP("notas")

# Las notas se guardan en notas.json, en la misma carpeta que este script
ARCHIVO = Path(__file__).parent / "notas.json"


# Lee el archivo de notas; si todavía no existe, devuelve una lista vacía
def cargar() -> list[dict]:
    if ARCHIVO.exists():
        return json.loads(ARCHIVO.read_text(encoding="utf-8"))
    return []


# Escribe la lista completa; ensure_ascii=False conserva tildes y eñes
def guardar(notas: list[dict]) -> None:
    ARCHIVO.write_text(
        json.dumps(notas, ensure_ascii=False, indent=2), encoding="utf-8"
    )


# ---------- Herramientas que verá el asistente ----------
@mcp.tool()
def guardar_nota(titulo: str, contenido: str) -> str:
    """Guarda una nota con título y contenido."""
    notas = cargar()
    notas.append({
        "titulo": titulo,
        "contenido": contenido,
        "fecha": datetime.now().isoformat(timespec="seconds"),
    })
    guardar(notas)
    return f"Nota '{titulo}' guardada. Total de notas: {len(notas)}"


@mcp.tool()
def listar_notas() -> list[dict]:
    """Devuelve todas las notas guardadas."""
    return cargar()


@mcp.tool()
def buscar_notas(texto: str) -> list[dict]:
    """Busca notas cuyo título o contenido contenga el texto indicado."""
    t = texto.lower()
    return [
        n for n in cargar()
        if t in n["titulo"].lower() or t in n["contenido"].lower()
    ]
```

```
# Punto de entrada: VS Code ejecuta este archivo y se comunica con él por stdio
if __name__ == "__main__":
    mcp.run()  # usa stdio por defecto
```

Lo importante del código:

- **@mcp.tool()** convierte una función normal en una herramienta MCP.
- El **docstring** es la descripción que lee el modelo para decidir cuándo usar la herramienta. Escríbelo claro.
- Los **tipos** (str, list[dict]) se convierten automáticamente en el esquema de parámetros de la herramienta.

Importante: no uses print() en un servidor stdio. La salida estándar está reservada para los mensajes del protocolo y cualquier texto adicional rompe la comunicación. Si necesitas depurar, escribe en la salida de errores con print(..., file=sys.stderr) o usa el Inspector del paso 9.

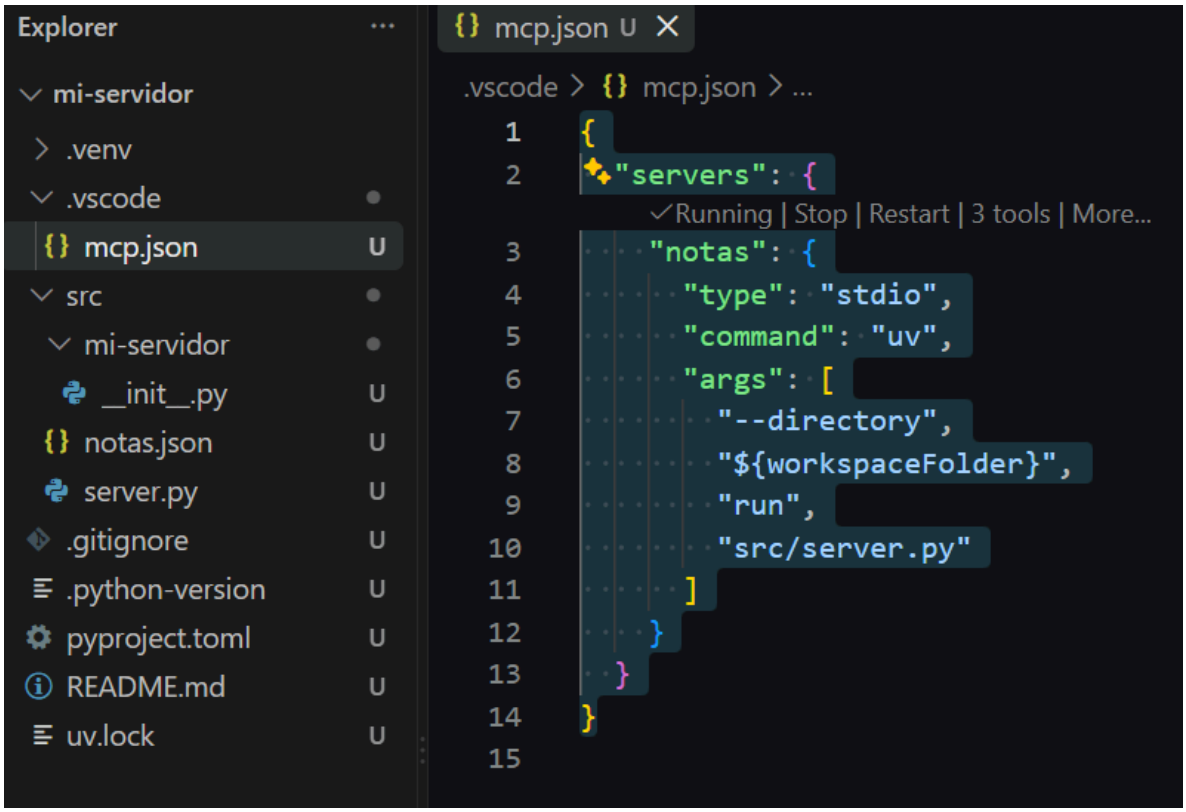
Paso 8: Registrar tu servidor en VS Code

Edita .vscode/mcp.json para que quede así:

```json
{
  "servers": {
    "notas": {
      "type": "stdio",
      "command": "uv",
      "args": [
        "--directory",
        "${workspaceFolder}",
        "run",
        "src/server.py"
      ]
    }
  }
}
```

La opción --directory le dice a uv que use el entorno virtual de mi-servidor, sin importar desde qué carpeta lo lance VS Code.

> **Consejo:** cada vez que modifiques server.py, reinicia el servidor (botón Restart sobre "notas", o MCP: List Servers → notas → Restart). VS Code no recarga el código por su cuenta.



Haz clic en ▶ **Start** sobre "notas" y pruébalo en el chat:

> *Guarda una nota titulada "Ideas" con el contenido "Aprender MCP con uv".*

Luego pide:

> *Busca mis notas sobre MCP.*

Qué pasa por dentro

Cuando le pides algo al asistente, la petición recorre estas etapas antes de que veas la respuesta:

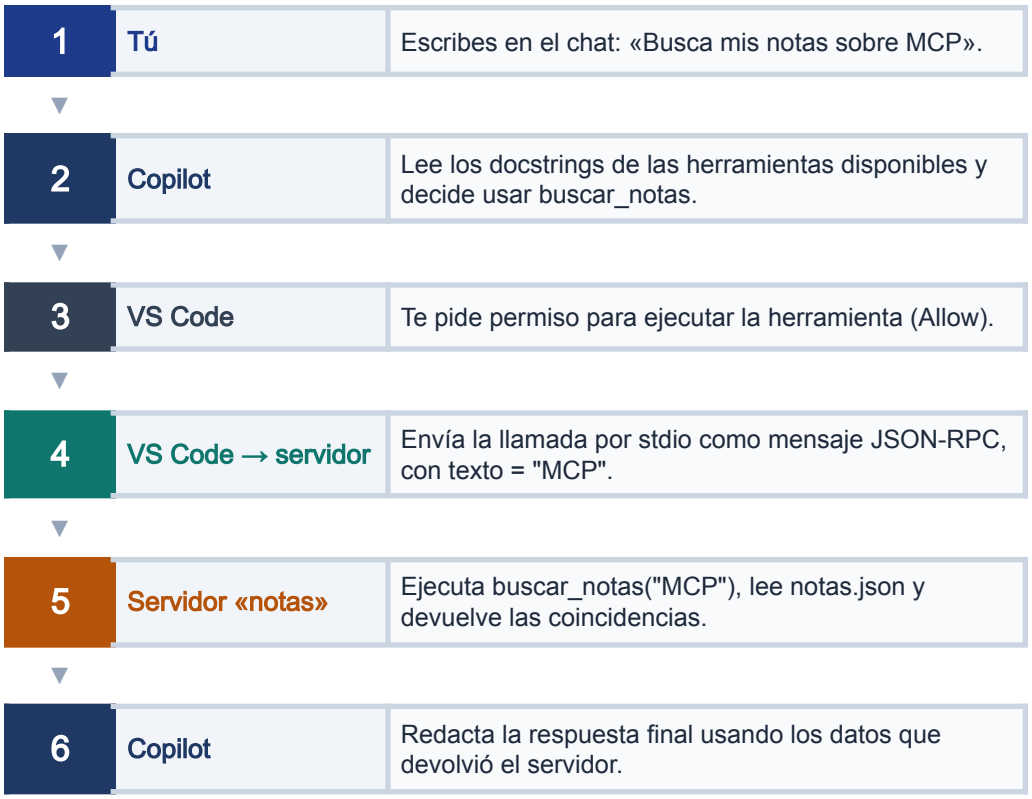


Figura 3. Qué ocurre por dentro cuando el asistente usa una herramienta MCP.

Paso 9: Depurar con el MCP Inspector (opcional, pero muy útil)

El Inspector es una interfaz web que te deja probar las herramientas de tu servidor sin pasar por Copilot. Para abrirlo, entra a mi-servidor y ejecuta:

```bash
uv run mcp dev src\server.py
```

Se abrirá el navegador. Pulsa **Connect**, ve a la pestaña **Tools** y ejecuta cada herramienta con los parámetros que quieras. Si algo falla ahí, el problema está en tu código y no en VS Code.

> **Nota:** el Inspector se ejecuta con Node por debajo; por eso lo instalaste en el paso 1. La primera vez puede pedirte confirmación para descargar el paquete del Inspector.

Paso 10 (extra): Usar servidores de terceros con uvx

Muchos servidores MCP escritos en Python se pueden ejecutar sin instalarlos, usando uvx. Por ejemplo, el servidor oficial **fetch**, que lee páginas web. Para usarlo, añade esto dentro de "servers":

```json
"fetch": {
  "type": "stdio",
  "command": "uvx",
  "args": ["mcp-server-fetch"]
}
```

Ojo con la coma: el bloque "fetch" va al mismo nivel que "memory" y "notas", y debe ir separado del anterior por una coma. Un JSON mal formado impide que arranque cualquiera de los servidores.

Solución de problemas comunes

Si algo no funciona, busca el síntoma en esta tabla antes de reinstalar nada:

Síntoma	Causa probable y solución
«npx» o «uv» no se reconoce como comando	La terminal o VS Code se abrieron antes de instalar. Ciérralos y vuelve a abrirlos; si persiste, reinicia el equipo.
No aparece el botón ▸ Start	El archivo no está en .vscode/mcp.json o el JSON tiene un error (una coma o una llave de más o de menos). Busca el subrayado rojo en el editor.
El servidor arranca, pero el chat no usa sus herramientas	No estás en modo Agent, o las herramientas están desactivadas en el icono 🔧 del chat.
El servidor «notas» se detiene al iniciar	Abre MCP: List Servers → notas → Show Output para ver el error. Suele ser un fallo de Python o un print() que rompe stdio.
La memoria no aparece en memoria/memory.jsonl	VS Code está abierto en otra carpeta y ${workspaceFolder} apunta a ella. Abre directamente proyecto-mcp.
Los cambios en server.py no se reflejan	El servidor sigue ejecutando la versión anterior. Reinícialo con Restart.